

GPU Computing

Exercise 7

Luis Wenzel
Miro Joensuu
Daniel Bayer
Noah Schneider

1 Presenting

Willing to present:

7.1 Yes

7.2

7.3

2 Naive GPU Implementation

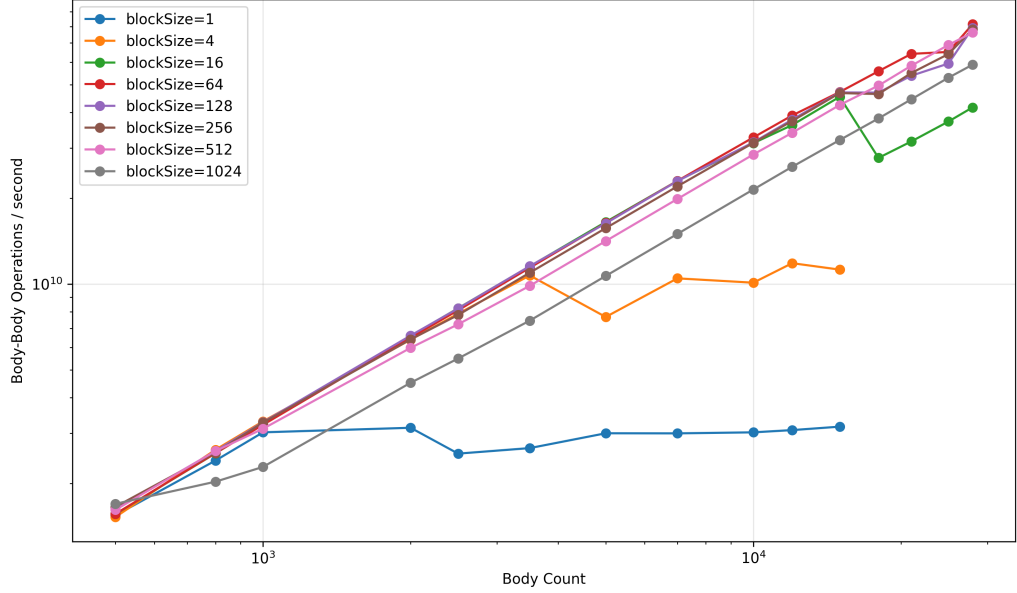


Figure 1: Naive GPU Performance (500 iterations)

Plot 1 shows the performance of the naive GPU implementation over 500 iterations. The measurements were taken as median values to account for outliers.

Performance increases with larger block sizes, as more warps can be scheduled concurrently. For block sizes of 256 and higher, performance is quite similar, as the GPU cores are well utilized. For smaller block sizes, performance drops significantly for larger matrices, since the CUDA cores are underutilized.

This means that the overall performance (interactions per second) also increases with a larger problem size, up to the point where a limit for the current block size is reached. From there, performance stays roughly constant.

We can also see that the larger block sizes (512 and 1024) perform slightly worse than the 64-256 range, which might be due to a reduced flexibility of scheduling.

3 Optimized GPU Implementation

With regards to using SOA vs AOS: It makes sense to use SOA for this use case, because when we fill the shared memory, we only need to read the position and mass data of each body. A SOA layout allows us to read only those properties, without having to deal with the velocity data, which would otherwise be interleaved in an AOS layout. There might be a penalty for the update of the position, since the position data is stored separately from the velocity data, but this will be negligible compared to the performance gain on the force calculation.

4 n-Body GPU computations and streaming